

38 Получение данных из Интернета

1. Прежде чем начать

Большинство представленных на рынке приложений Android подключаются к Интернету для выполнения сетевых операций, таких как получение электронных писем, сообщений или другой информации с внутреннего сервера. Gmail, YouTube и Google Photos — это примеры приложений, которые подключаются к Интернету для отображения пользовательских данных.

В этой лабораторной вы будете использовать библиотеки с открытым исходным кодом и библиотеки, управляемые сообществом, для создания уровня данных и получения данных с внутреннего сервера. Это значительно упрощает получение данных, а также помогает приложению следовать лучшим практикам Android, например выполнять операции в фоновом потоке. Вы также отобразите сообщение об ошибке, если Интернет работает медленно или недоступен, что будет информировать пользователя о любых проблемах с сетевым подключением.

Что вы узнаете

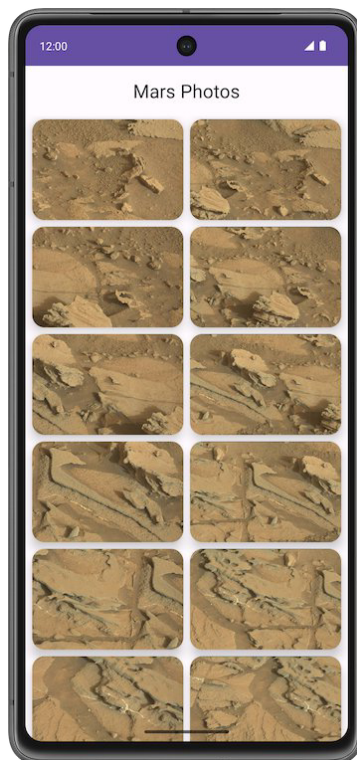
- Что такое веб-сервис [REST](#).
- Как использовать библиотеку [Retrofit](#) для подключения к веб-службе REST в Интернете и получения ответа.
- Как использовать библиотеку [сериализации \(kotlinx.serialization\)](#) для анализа ответа JSON в объект данных.

Что ты будешь делать

- Измените начальное приложение, чтобы оно отправляло запрос API веб-службы и обрабатывало ответ.
- Реализуйте уровень данных для своего приложения с помощью библиотеки Retrofit.
- Проанализируйте ответ JSON от веб-службы в список объектов данных вашего приложения с помощью библиотеки *kotlinx.serialization* и прикрепите его к состоянию пользовательского интерфейса.
- Используйте поддержку сопрограмм Retrofit для упрощения кода.

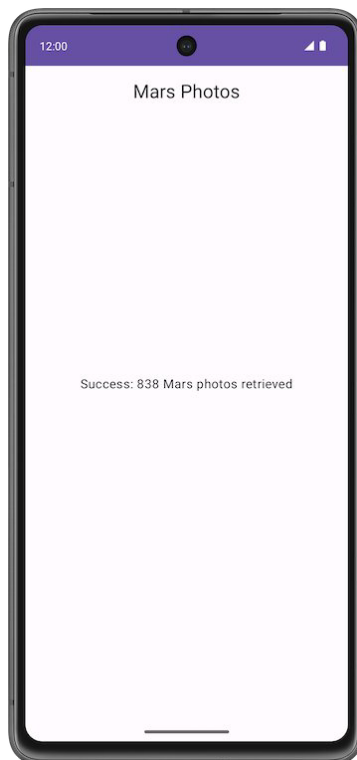
2. Обзор приложения

Вы работаете с приложением **Mars Photos**, которое показывает изображения поверхности Марса. Это приложение подключается к веб-сервису для получения и отображения фотографий Марса. Изображения представляют собой реальные фотографии с Марса, снятые с марсоходов НАСА. Следующее изображение представляет собой снимок экрана финальной версии приложения, которое содержит сетку изображений.



Примечание . Предыдущее изображение представляет собой снимок экрана окончательного приложения, которое вы создадите в конце этого модуля после дополнительных обновлений в последующих лабораториях кода. Снимок экрана показан в этой лаборатории кода, чтобы дать вам лучшее представление об общей функциональности приложения.

Версия приложения, которое вы создадите в этой лаборатории кода, не будет иметь большого количества визуальных эффектов. В этой лабораторной основное внимание уделяется части уровня данных приложения для подключения к Интернету и загрузки необработанных данных о свойствах с помощью веб-службы. Чтобы приложение правильно извлекало и анализировало эти данные, вы можете распечатать количество фотографий, полученных с внутреннего сервера, в составном элементе `Text`.



3. Изучите стартовое приложение Mars Photos.

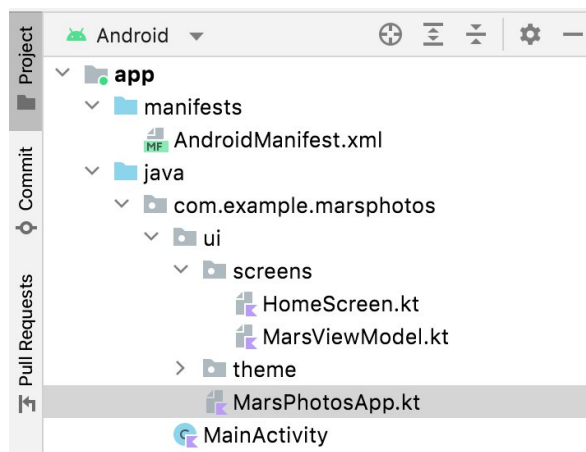
Скачать стартовый код

Для начала загрузите стартовый код:

[file_download](#) [Скачать zip](#)

Запустить стартовый код

1. Откройте загруженный проект в Android Studio. Имя папки проекта — `basic-android-kotlin-compose-training-mars-photos`.
2. На панели **Android** разверните **приложение** > **Java** . Обратите внимание, что в приложении есть папка пакета с именем `ui`. Это уровень пользовательского интерфейса приложения.



3. Запустите приложение. Когда вы скомпилируете и запустите приложение, вы увидите следующий экран с текстом-заполнителем в центре. В конце этой лабораторной вы обновите этот текст-заполнитель, указав количество полученных фотографий.



Пошаговое руководство по стартовому коду

В этом задании вы ознакомитесь со структурой проекта. В следующих списках представлены важные файлы и папки проекта.

`ui\MarsPhotosApp.kt`:

- Этот файл содержит составной объект `MarsPhotosApp`, который отображает содержимое на экране, например верхнюю панель приложения и составной элемент `HomeScreen`. Текст-заполнитель из предыдущего шага отображается в этом компоновочном элементе.
- В следующей лабораторной составная часть отображает данные, полученные с внутреннего сервера фотографий Марса.

`screens\MarsViewModel.kt`:

- Этот файл представляет собой соответствующую модель представления для `MarsPhotosApp`.
- Этот класс содержит `MutableState` свойство с именем `marsUiState`. Обновление значения этого свойства обновляет текст заполнителя, отображаемый на экране.
- Метод `getMarsPhotos()` обновляет ответ заполнителя. Позже вы будете использовать этот метод для отображения данных, полученных с сервера. Целью этой лабораторной является обновление `MutableState` данных `ViewModel` об использовании, которые вы получаете из Интернета.

`screens\HomeScreen.kt`:

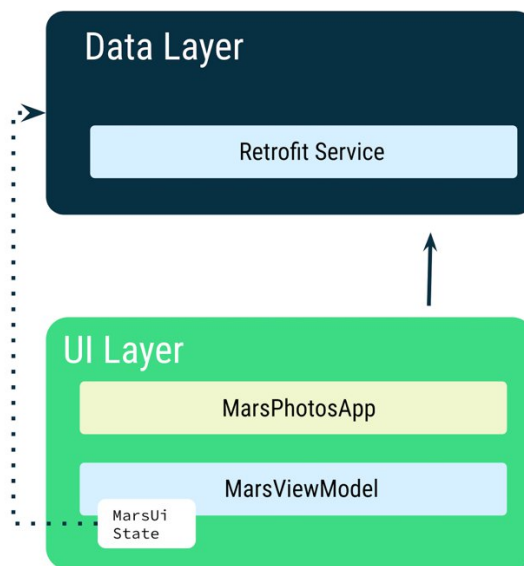
- Этот файл содержит элементы `HomeScreen` и `ResultScreen` составные элементы. `ResultScreen` имеет простой макет `Box`, который отображает значение `marsUiState` в составном элементе `Text`.

MainActivity.kt:

- Единственная задача этого действия — загрузить `ViewModel` и отобразить составной элемент `MarsPhotosApp`.

4. Введение в веб-сервисы

В этой лабораторной вы создаете уровень для сетевой службы, которая взаимодействует с внутренним сервером и извлекает необходимые данные. Для реализации этой задачи вы используете стороннюю библиотеку [Retrofit](#). Подробнее об этом вы узнаете позже. Он `ViewModel` взаимодействует с уровнем данных, а остальная часть приложения прозрачна для этой реализации.

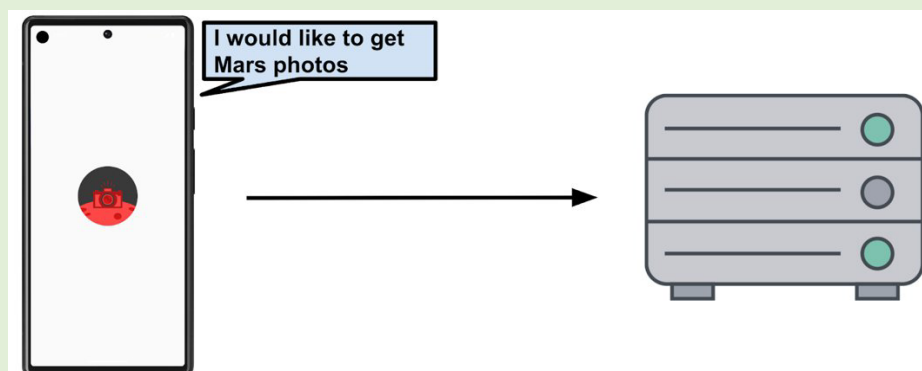


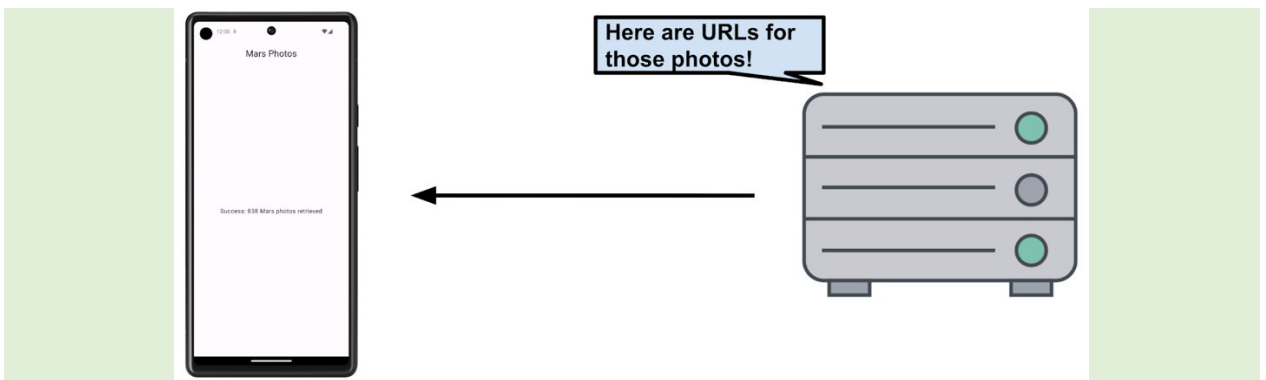
`MarsViewModel` отвечает за сетевой вызов для получения данных фотографий Марса. В `ViewModel` вы используете `MutableState` для обновления пользовательского интерфейса приложения при изменении данных.

Примечание. В последующих лабораторных работах вы добавите репозиторий на свой уровень данных. Затем репозиторий связывается со службой `Retrofit` для получения данных. Репозиторий отвечает за предоставление данных остальной части приложения.

5. Веб-сервисы и модернизация

Данные фотографий Марса хранятся на веб-сервере. Чтобы перенести эти данные в ваше приложение, вам необходимо установить соединение и связаться с сервером в Интернете.





Примечание . В этой кодовой лаборатории вы получаете только количество записей (URL-адресов), а не фотографии Марса. В более поздней кодовой лаборатории вы извлекаете фотографии Марса и отображаете их в сетке.

Большинство веб-серверов сегодня запускают веб-сервисы , используя общую веб-архитектуру без сохранения состояния, известную как [REST](#) , что означает **RE** Presental **State** Transfer. Веб-сервисы, предлагающие эту архитектуру, известны как сервисы RESTful.

Запросы к веб-службам RESTful выполняются стандартизированным способом через [унифицированные идентификаторы ресурсов](#) (URI). URI идентифицирует ресурс на сервере по имени, не указывая его местоположение или способ доступа к нему. Например, в приложении для этого урока вы получаете URL-адреса изображений, используя следующий URI сервера. (На этом сервере размещены как марсианская недвижимость, так и фотографии Марса):

android-kotlin-fun-mars-server.appspot.com

Примечание . К вышеупомянутому серверу обращается другой пример приложения, в котором оно демонстрирует недвижимость на Марсе, поэтому у этого сервера есть две разные конечные точки: одна для недвижимости на Марсе и одна для фотографий. В этом курсе вы будете использовать сервер для получения фотографий Марса.

URL-адрес (унифицированный указатель ресурса) — это подмножество URI, которое указывает, где находится ресурс, и механизм его получения.

Например:

Следующий URL-адрес дает список доступных объектов недвижимости на Марсе:

<https://android-kotlin-fun-mars-server.appspot.com/realestate>

По следующему URL-адресу можно получить список фотографий Марса:

<https://android-kotlin-fun-mars-server.appspot.com/photos>

Эти URL-адреса относятся к определенному ресурсу, например [/realestate](#) или [/photos](#) , который можно получить через протокол передачи гипертекста (*http:*) из сети. В этой лабораторной вы используете конечную точку [/photos](#) . Конечная точка — это URL-адрес, который позволяет вам получить доступ к веб-службе, работающей на сервере.

Примечание . Знакомый веб-URL на самом деле является разновидностью URI. В этом курсе URL и URI используются взаимозаменяемо.

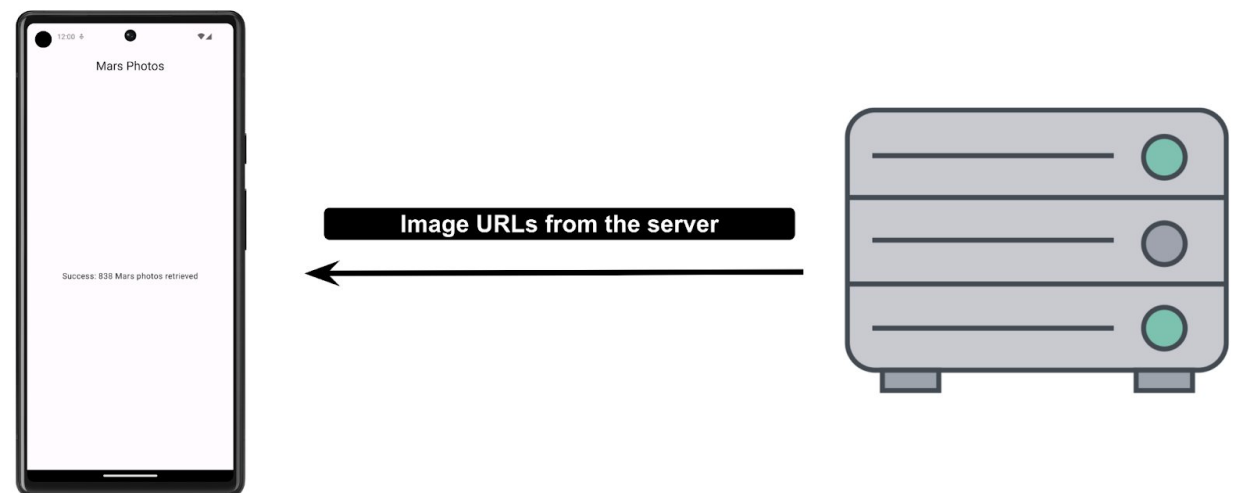
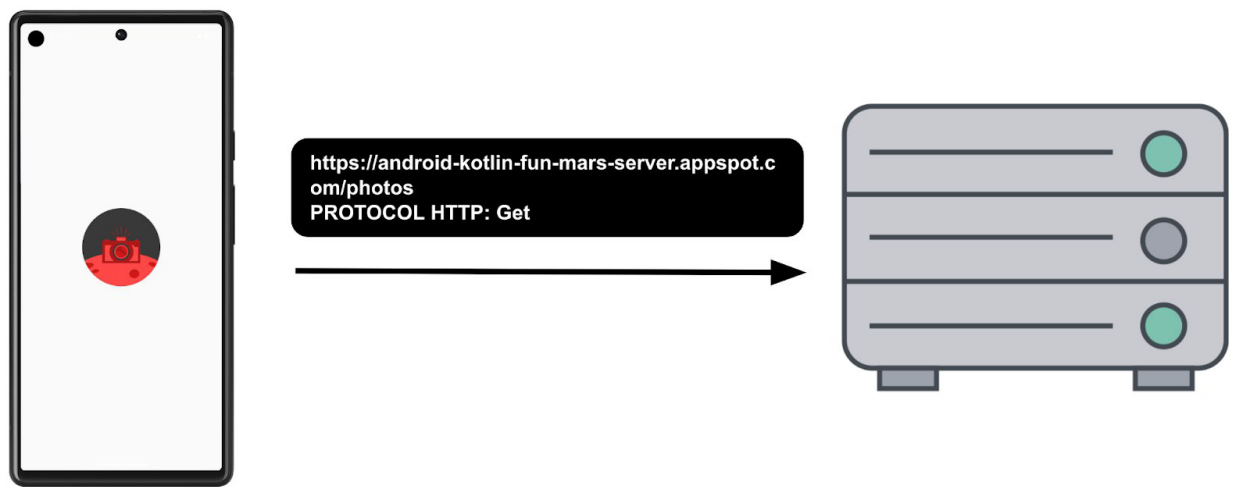
Запрос веб -сервиса

Каждый запрос веб-службы содержит URI и передается на сервер по тому же протоколу HTTP, который используется веб-браузерами, например Chrome. HTTP-запросы содержат операцию, сообщаящую серверу, что делать.

Общие операции HTTP включают в себя:

- GET для получения данных с сервера.
- POST для создания новых данных на сервере.
- PUT для обновления существующих данных на сервере.
- DELETE для удаления данных с сервера.

Ваше приложение отправляет HTTP-запрос GET на сервер для получения информации о фотографиях Марса, а затем сервер возвращает ответ вашему приложению, включая URL-адреса изображений.



Ответ веб-службы форматируется в одном из распространенных форматов данных, например XML (расширяемый язык разметки) или JSON (нотация объектов JavaScript). Формат JSON представляет структурированные данные в парах ключ-значение. Приложение взаимодействует с REST API с помощью JSON, о котором вы узнаете больше в следующем задании.

В этой задаче вы устанавливаете сетевое соединение с сервером, взаимодействуете с сервером и получаете ответ в формате JSON. Вы будете использовать серверный сервер,

который уже написан для вас. В этой лабораторной вы используете библиотеку Retrofit, стороннюю библиотеку, для связи с внутренним сервером.

Внешние библиотеки

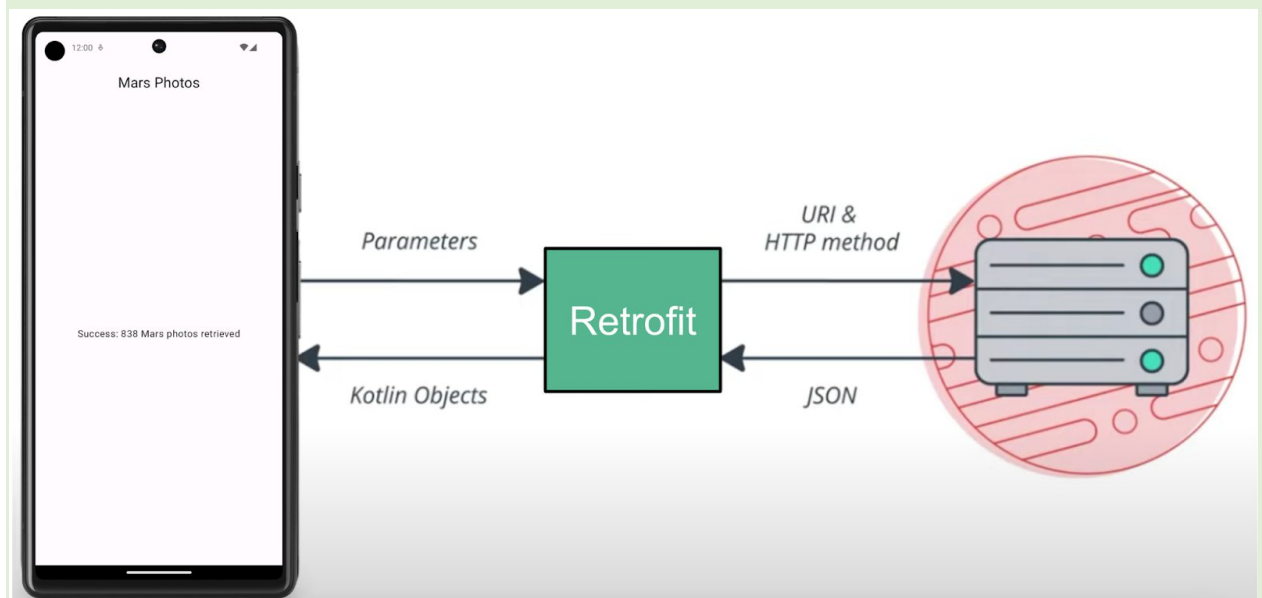
Внешние библиотеки или сторонние библиотеки подобны расширениям основных API-интерфейсов Android. Библиотеки, которые вы используете в этом курсе, имеют открытый исходный код, разработаны сообществом и поддерживаются коллективным вкладом огромного сообщества Android по всему миру. Эти ресурсы помогают таким разработчикам Android, как вы, создавать более качественные приложения.

Предупреждение: использование библиотек, разработанных и поддерживаемых сообществом, может значительно сэкономить время. Однако вы должны выбирать эти библиотеки с умом, поскольку ваше приложение в конечном итоге несет ответственность за то, что делает код в этих библиотеках.

Модернизация библиотеки

Библиотека Retrofit, которую вы используете в этой лабораторной для взаимодействия с веб-службой RESTful Mars, является хорошим примером хорошо поддерживаемой и поддерживаемой библиотеки. Вы можете убедиться в этом, посмотрев его страницу [GitHub](#) и просмотрев открытые и закрытые проблемы (некоторые из них являются запросами функций). Если разработчики регулярно решают проблемы и отвечают на запросы функций, скорее всего, библиотека поддерживается в хорошем состоянии и является хорошим кандидатом для использования в приложении. Вы также можете обратиться к документации [Retrofit](#), чтобы узнать больше о библиотеке.

Библиотека Retrofit взаимодействует с серверной частью REST. Он генерирует код, но вам необходимо предоставить URI для веб-сервиса на основе параметров, которые мы ему передаем. Вы узнаете больше об этой теме в последующих разделах.



Добавить зависимости для модернизации

Android Gradle позволяет добавлять в проект внешние библиотеки. Помимо зависимости библиотеки, вам также необходимо указать репозиторий, в котором размещена библиотека.

1. Откройте файл Gradle уровня модуля `build.gradle.kts` (Module :app).

2. В разделе `dependencies` добавьте следующие строки для библиотек Retrofit:

```
// Retrofit
implementation("com.squareup.retrofit2:retrofit:2.9.0")
// Retrofit with Scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

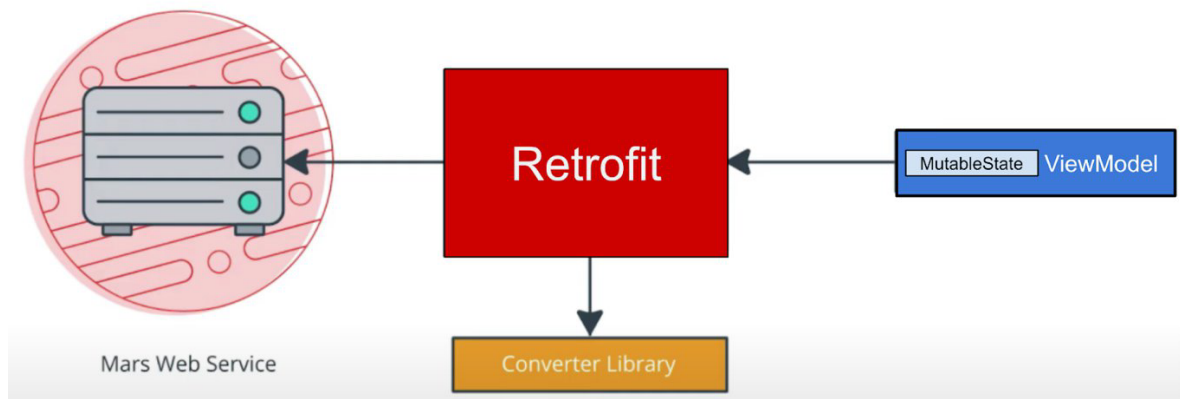
Две библиотеки работают вместе. Первая зависимость предназначена для самой библиотеки Retrofit2, а вторая зависимость — для скалярного преобразователя Retrofit. Retrofit2 — это обновленная версия библиотеки Retrofit. Этот скалярный преобразователь позволяет Retrofit возвращать результат JSON в виде файла `String`. JSON — это формат хранения и передачи данных между клиентом и сервером.

3. Нажмите **«Sync Now»**, чтобы перестроить проект с новыми зависимостями.

6. Подключение к Интернету

Вы используете библиотеку Retrofit для взаимодействия с веб-сервисом Mars и отображения необработанного ответа JSON в виде файла `String`. Заполнитель `Text` отображает либо возвращенную строку ответа JSON, либо сообщение, указывающее на ошибку соединения.

Retrofit создает сетевой API для приложения на основе содержимого веб-службы. Он извлекает данные из веб-сервиса и направляет их через отдельную библиотеку конвертеров, которая умеет декодировать данные и возвращать их в виде объектов, например `String`. Retrofit включает встроенную поддержку популярных форматов данных, таких как XML и JSON. В конечном итоге Retrofit создает для вас код для вызова и использования этой службы, включая важные детали, такие как выполнение запросов в фоновых потоках.

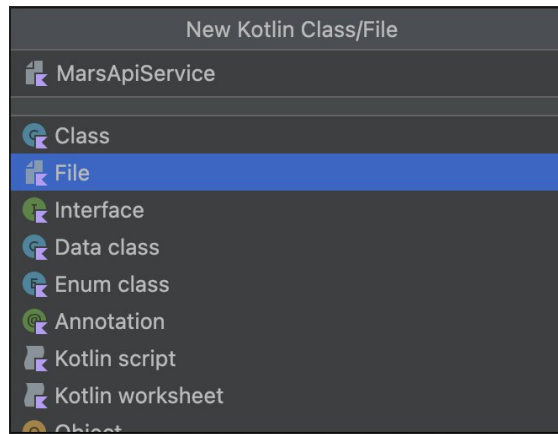


В этой задаче вы добавляете в проект Mars Photos слой данных `ViewModel`, который используете для связи с веб-сервисом. Вы реализуете API службы Retrofit, выполнив следующие шаги:

- Создадите источник данных, класс `MarsApiService`.
- Создадите объект Retrofit с базовым URL-адресом и фабрикой преобразователя для преобразования строк.
- Создадите интерфейс, объясняющий, как Retrofit взаимодействует с веб-сервером.
- Создадите службу Retrofit и предоставите экземпляр службе API остальной части приложения.

Выполните вышеуказанные шаги:

1. Щелкните правой кнопкой мыши пакет `com.example.marsphotos` на панели проекта Android и выберите **«New» > «Package»**.
2. Во всплывающем окне добавьте в конец предложенного имени пакета название папки **network**.
3. Создайте новый файл Kotlin в новом пакете. Назови это `MarsApiService`.



4. Откроется `network/MarsApiService.kt`.
5. Добавьте следующую константу для базового URL-адреса веб-службы.

```
private const val BASE_URL = "https://android-kotlin-fun-mars-server.appspot.com"
```

6. Добавьте построитель Retrofit чуть ниже этой константы, чтобы построить и создать объект Retrofit.

```
import retrofit2.Retrofit
```

```
private val retrofit = Retrofit.Builder()
```

Retrofit необходим базовый URI для веб-службы и фабрика преобразователей для создания API веб-служб. Конвертер сообщает Retrofit, что делать с данными, которые он получает обратно от веб-сервиса. В этом случае вы хотите, чтобы Retrofit получал ответ JSON от веб-службы и возвращал его в виде файла `String`. Retrofit имеет объект `ScalarsConverter`, поддерживающий строки и другие примитивные типы.

7. Вызовите `addConverterFactory()` сборщика с экземпляром `ScalarsConverterFactory`.

```
import retrofit2.converter.scalars.ScalarsConverterFactory
```

```
private val retrofit = Retrofit.Builder()
```

```
    .addConverterFactory(ScalarsConverterFactory.create())
```

8. Добавим базовый URL-адрес веб-службы, используя метод `baseUrl()`. Вызовите `build()` для создания объекта Retrofit.

```
private val retrofit = Retrofit.Builder()
```

```
    .addConverterFactory(ScalarsConverterFactory.create())
```

```
    .baseUrl(BASE_URL)
```

```
    .build()
```

10. Под вызовом конструктора Retrofit определите интерфейс `MarsApiService`, который определяет, как Retrofit взаимодействует с веб-сервером с помощью HTTP-запросов.

```
interface MarsApiService {  
}
```

11. Добавьте в интерфейс функцию `getPhotos()`, вызываемую `MarsApiService` для получения строки ответа от веб-службы.

```
interface MarsApiService {  
    fun getPhotos()  
}
```

12. Используйте аннотацию `@GET`, чтобы сообщить Retrofit, что это запрос GET, и укажите конечную точку для этого метода веб-службы. В данном случае конечная точка `photos`. В этой лабораторной вы будете использовать конечную точку [/photos](#).

```
import retrofit2.http.GET
```

```
interface MarsApiService {  
    @GET("photos")  
    fun getPhotos()  
}
```

При вызове метода `getPhotos()` Retrofit добавляет конечную точку `photos` к базовому URL-адресу, который вы определили в построителе Retrofit, который используется для запуска запроса.

13. Добавьте тип `String` возвращаемого значения функции.

```
interface MarsApiService {  
    @GET("photos")  
    fun getPhotos(): String  
}
```

Объявления объектов

В Kotlin [объявления объектов](#) используются для объявления одноэлементных объектов. [Шаблон Singleton](#) гарантирует, что создается один и только один экземпляр объекта и имеется одна глобальная точка доступа к этому объекту. Инициализация объекта является потокобезопасной и выполняется при первом доступе.

Ниже приведен пример объявления объекта и доступа к нему. Объявление объекта всегда имеет имя, следующее за `object` ключевым словом.

Пример:

// Пример для объявления объекта, не копируйте

```
object SampleDataProvider {  
    fun register(provider: SampleProvider) {  
        // ...  
    }  
}
```

```
// ...  
}
```

// Чтобы обратиться к объекту, используйте непосредственно его название.
SampleDataProvider.register(...)

Предупреждение . Использование шаблона Singleton не рекомендуется. Синглтоны представляют собой глобальные состояния, которые трудно предсказать, особенно в тестах. Объекты должны определять, какие зависимости им нужны, а не описывать, как их создавать.

Используйте [внедрение зависимостей](#) вместо одноэлементного шаблона. В более поздней лаборатории вы узнаете, как реализовать внедрение зависимостей.

Вызов функции `create()` объекта Retrofit требует больших затрат памяти, скорости и производительности. Приложению нужен *только один* экземпляр службы Retrofit API, поэтому вы предоставляете службу остальной части приложения с помощью *объявления объекта* .

1. За пределами объявления интерфейса `MarsApiService` определите общедоступный объект, вызываемый `MarsApi` для инициализации службы Retrofit. Этот объект является общедоступным одноэлементным объектом, к которому имеет доступ остальная часть приложения.

```
object MarsApi {}
```

2. Внутри объявления объекта `MarsApi` добавьте *lazy* инициализированное модифицированное свойство объекта с именем `retrofitService` типа `MarsApiService`. Вы выполняете эту *lazy* инициализацию, чтобы убедиться, что она инициализируется при первом использовании. Пройгнорируйте ошибку, которую вы исправите на следующих шагах.

Примечание . Помните, что «*lazy* инициализация» — это когда создание объекта намеренно откладывается до тех пор, пока этот объект действительно вам не понадобится, чтобы избежать ненужных вычислений или использования других вычислительных ресурсов. Котлин имеет [первоклассную поддержку](#) ленивого создания экземпляров.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {}  
}
```

3. Инициализируйте переменную `retrofitService`, используя метод `retrofit.create()` с интерфейсом `MarsApiService`.

```
object MarsApi {  
    val retrofitService : MarsApiService by lazy {  
        retrofit.create(MarsApiService::class.java)  
    }  
}
```

Настройка дооснащения завершена! Каждый раз, когда ваше приложение вызывает `MarsApi.retrofitService`, вызывающая сторона обращается к тому же одноэлементному объекту Retrofit, который реализует `MarsApiService`, который создается при первом доступе. В следующей задаче вы используете реализованный вами объект Retrofit.

Вызов веб-сервиса в MarsViewModel

На этом этапе вы реализуете метод `getMarsPhotos()`, который вызывает службу REST, а затем обрабатывает возвращенную строку JSON.

Примечание. Рекомендуемый подход - вызов веб-сервиса из репозитория, который изолирует уровень данных от остальной части приложения. В более поздней лабораторной вы добавите репозиторий в свое приложение.

ViewModelScope

`viewModelScope` — это встроенная область действия сопрограммы (корутины), определенная для каждого приложения `ViewModel` в вашем приложении. Любая сопрограмма, запущенная в этой области, автоматически отменяется, если очищается `ViewModel`.

Вы можете использовать `viewModelScope` для запуска сопрограммы (корутины) и выполнения запроса веб-службы в фоновом режиме. Поскольку `viewModelScope` принадлежит `ViewModel`, запрос продолжается, даже если в приложении происходит изменение конфигурации.

1. В файле `MarsApiService.kt` измените функцию `getPhotos()`, чтобы сделать ее асинхронной и не блокировать вызывающий поток. Вы вызываете эту функцию изнутри файла `viewModelScope`.

```
@GET("photos")
```

```
suspend fun getPhotos(): String
```

2. Откройте файл `ui/screens/MarsViewModel.kt`. Прокрутите вниз до метода `getMarsPhotos()`. Удалите строку, которая устанавливает ответ состояния `"Set the Mars API Response here!"`, чтобы метод `getMarsPhotos()` был пустым.

```
private fun getMarsPhotos() {}
```

3. Внутри `getMarsPhotos()` запустите сопрограмму, используя `viewModelScope.launch`.

```
import androidx.lifecycle.viewModelScope
```

```
import kotlinx.coroutines.launch
```

```
private fun getMarsPhotos() {  
    viewModelScope.launch {}  
}
```

4. Внутри `viewModelScope` используйте объект `Singleton MarsApi` для вызова метода `getPhotos()` из интерфейса `retrofitService`. Сохраните возвращенный ответ `val` в вызываемом файле `listResult`.

```
import com.example.marsphotos.network.MarsApi
```

```
viewModelScope.launch {  
    val listResult = MarsApi.retrofitService.getPhotos()  
}
```

5. Назначьте результат, только что полученный от внутреннего сервера, файлу `marsUiState`. Это изменяемый объект состояния `marsUiState`, который представляет состояние самого последнего веб-запроса.

```
val listResult = MarsApi.retrofitService.getPhotos()
marsUiState = listResult
```

6. Запустите приложение. Обратите внимание, что приложение немедленно закрывается и может отображать или не отображать всплывающее окно с ошибкой. Это сбой приложения.
7. Перейдите на вкладку **Logcat** в Android Studio и обратите внимание на ошибку в журнале, которая начинается со следующей строки: "----- beginning of crash"

```
----- beginning of crash
22803-22865/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION: OkHttp
Dispatcher
Process: com.example.android.marsphotos, PID: 22803
java.lang.SecurityException: Permission denied (missing INTERNET permission?)
...
```

Это сообщение об ошибке указывает на то, что приложению, возможно, не хватает **INTERNET** разрешений. Следующая задача описывает, как добавить приложению разрешения на доступ в Интернет и решить эту проблему.

7. Добавьте разрешение Интернета и обработку исключений.

Разрешения Android

Целью разрешений на Android является защита конфиденциальности пользователя Android. Приложения Android должны заявлять или запрашивать разрешения на доступ к конфиденциальным данным пользователя, таким как контакты, журналы вызовов и определенные функции системы, такие как камера или Интернет.

Чтобы ваше приложение могло получить доступ к Интернету, ему необходимо разрешение **INTERNET**. Подключение к Интернету создает проблемы безопасности, поэтому приложения по умолчанию не имеют подключения к Интернету. Вам необходимо явно указать, что приложению необходим доступ к Интернету. Это заявление считается обычным разрешением. Дополнительную информацию о разрешениях Android и их типах см. в разделе [Разрешения на Android](#).

На этом этапе ваше приложение объявляет необходимые ему разрешения, включая теги `<uses-permission>` в файл `AndroidManifest.xml`.

1. Откройте `manifests/AndroidManifest.xml`. Добавьте эту строку непосредственно перед `<application>` тегом:

```
<uses-permission android:name="android.permission.INTERNET" />
```

2. Скомпилируйте и запустите приложение еще раз.

Если у вас есть работающее подключение к Интернету, вы увидите текст JSON, содержащий данные, связанные с фотографиями Марса. Обратите внимание, как `id` и `img_src` повторяются для каждой записи изображения. Подробнее о формате JSON вы узнаете позже.



3. Нажмите кнопку **«Back»** на своем устройстве или в эмуляторе, чтобы закрыть приложение.

Обработка исключений

В вашем коде есть ошибка. Чтобы увидеть ее, выполните следующие действия:

1. Переведите свое устройство или эмулятор в режим полета, чтобы имитировать ошибку сетевого подключения.
2. Снова откройте приложение из меню «Последние» или запустите его из Android Studio.
3. Перейдите на вкладку **Logcat** в Android Studio и обратите внимание на фатальное исключение в журнале, которое выглядит следующим образом:

```
3302-3302/com.example.android.marsphotos E/AndroidRuntime: FATAL EXCEPTION: main
Process: com.example.android.marsphotos, PID: 3302
```

Это сообщение об ошибке указывает на то, что приложение пыталось подключиться, но время ожидания истекло. Подобные исключения очень распространены в реальном времени. В отличие от проблемы с разрешениями, эту ошибку нельзя исправить, но вы можете с ней справиться. На следующем шаге вы узнаете, как обрабатывать такие исключения.

Исключения

Исключениями являются ошибки, которые могут возникнуть во время выполнения, а не во время компиляции, и они внезапно завершают работу приложения без уведомления пользователя. Это может привести к ухудшению пользовательского опыта. *Обработка исключений* — это механизм, с помощью которого вы предотвращаете внезапное завершение работы приложения и решаете ситуацию удобным для пользователя способом.

Причиной исключений может быть простое деление на ноль или ошибка сетевого подключения. Эти исключения аналогичны тем `IllegalArgumentException`, которые обсуждались в предыдущей лаборатории.

Примеры потенциальных проблем при подключении к серверу включают следующее:

- URL-адрес или URI, используемый в API, неверен.
- Сервер недоступен, и приложению не удалось подключиться к нему.
- Проблема с задержкой сети.
- Плохое подключение к Интернету или его отсутствие на устройстве.

Эти исключения не могут быть обработаны во время компиляции, но вы можете использовать блок `try-catch` для обработки исключения во время выполнения. Для получения дополнительной информации обратитесь к [Исключениям](#).

Пример синтаксиса для блока `try-catch`

```
try {
    // some code that can cause an exception.
}
catch (e: SomeException) {
    // handle the exception to avoid abrupt termination.
}
```

Внутри блока `try` вы добавляете код, в котором ожидаете исключение. В вашем приложении это сетевой вызов. В блоке `catch` необходимо реализовать код, предотвращающий резкое завершение работы приложения. Если есть исключение, то блок `catch` выполняется для восстановления после ошибки, а не резко завершает работу приложения.

1. В файле `MarsViewModel` внутри блока `launch` добавьте блок `try` вокруг вызова `MarsApi` для обработки исключений. Добавьте блок `catch` после блока `try`.

```
import java.io.IOException

...
viewModelScope.launch {
    try {
        val listResult = MarsApi.retrofitService.getPhotos()
        marsUiState = listResult
    } catch (e: IOException) {

    }
}
```

3. Запустите приложение еще раз. Обратите внимание, что на этот раз приложение не аварийно завершает работу.

Добавить пользовательский интерфейс

В классе `MarsViewModel` статус самого последнего веб-запроса `marsUiState` сохраняется как изменяемый объект состояния. Однако в этом классе отсутствует возможность сохранять различные состояния: загрузка, успех и сбой.

- **Статус загрузки** указывает, что приложение ожидает данных.
- **Статус успеха** указывает, что данные были успешно получены из веб-службы.
- **Статус ошибки** указывает на любые ошибки сети или соединения.

Чтобы представить эти три состояния в вашем приложении, вы используете *sealed interface*, который упрощает управление состоянием, ограничивая возможные значения. В приложении Mars Photos вы ограничиваете `marsUiState` веб-ответ тремя состояниями (объектами класса данных): загрузка, успех и ошибка, что выглядит следующим образом:

// не нужно копировать

```
sealed interface MarsUiState {  
    data class Success : MarsUiState  
    data class Loading : MarsUiState  
    data class Error : MarsUiState  
}
```

В приведенном выше фрагменте кода в случае успешного ответа вы получаете с сервера информацию о фотографии Марса. Чтобы сохранить данные, добавьте параметр конструктора в класс данных `Success`.

В случае состояний `Loading` и `Error` вам не нужно задавать новые данные и создавать новые объекты; вы просто передаете веб-ответ. Измените `data` класс на `Object`, чтобы создать объекты для веб-ответов.

1. Откройте файл `ui/MarsViewModel.kt`. После операторов импорта добавьте `sealed interface MarsUiState`. Благодаря этому добавлению объект значений `MarsUiState` может иметь исчерпывающий характер.

```
sealed interface MarsUiState {  
    data class Success(val photos: String) : MarsUiState  
    object Error : MarsUiState  
    object Loading : MarsUiState  
}
```

2. Внутри класса `MarsViewModel` обновите определение `marsUiState`. Измените тип на `MarsUiState` и `MarsUiState.Loading` как значение по умолчанию. Сделайте установщик `private`, чтобы защитить запись в файл `marsUiState`.

```
var marsUiState: MarsUiState by mutableStateOf(MarsUiState.Loading)  
private set
```

3. Прокрутите вниз до метода `getMarsPhotos()`. Обновите значение `marsUiState` `MarsUiState.Success` и передайте `listResult`.

```
val listResult = MarsApi.retrofitService.getPhotos()  
marsUiState = MarsUiState.Success(listResult)
```

4. Внутри блока `catch` обработайте ответ об ошибке. Установим `MarsUiState` в `Error`.

```
catch (e: IOException) {  
    marsUiState = MarsUiState.Error  
}
```

5. Вы можете снять задание `marsUiState` из блока `try-catch`. Завершенная функция должна выглядеть следующим образом:

```

fun getMarsPhotos() {
    viewModelScope.launch {
        marsUiState = try {
            val listResult = MarsApi.retrofitService.getPhotos()
            MarsUiState.Success(listResult)
        } catch (e: IOException) {
            MarsUiState.Error
        }
    }
}

```

6. В файл `screens/HomeScreen.kt` добавьте выражение `when` для `marsUiState`. Если `marsUiState = MarsUiState.Success`, вызываем `ResultScreen` и отправляем `marsUiState.photos`. **Пока игнорируйте ошибки.**

```

import androidx.compose.foundation.layout.fillMaxWidth

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
){
    when (marsUiState) {
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )
    }
}

```

Примечание. Свойство `marsUiState` больше не является строкой. Вы изменили его на sealed interface `MarsUiState`, который может иметь три разных значения объекта: `MarsUiState.Loading`, `MarsUiState.Success` и `MarsUiState.Error`.

7. Внутри блока `when` добавьте проверки на `MarsUiState.Loading` и `MarsUiState.Error`. Пусть приложение отображает составные объекты `LoadingScreen`, `ResultScreen` и `ErrorScreen`, которые вы реализуете позже.

```

import androidx.compose.foundation.layout.fillMaxSize

fun HomeScreen(
    marsUiState: MarsUiState,
    modifier: Modifier = Modifier
){
    when (marsUiState) {
        is MarsUiState.Loading -> LoadingScreen(modifier = modifier.fillMaxSize())
        is MarsUiState.Success -> ResultScreen(
            marsUiState.photos, modifier = modifier.fillMaxWidth()
        )

        is MarsUiState.Error -> ErrorScreen(modifier = modifier.fillMaxSize())
    }
}

```

```
}
```

Примечание . Если вы реализуете `MarsUiState` interface без ключевого слова `sealed`, вам потребуется добавить `Success`, `Error`, `Loading` и ветвь `else`. Поскольку четвертого варианта (иначе) нет, вы используете интерфейс `sealed`, чтобы сообщить компилятору, что существует только три варианта (таким образом делая условные выражения исчерпывающими).

9. В файле `screens/HomeScreen.kt` под составным элементом `HomeScreen` добавьте следующую составную функцию `LoadingScreen` для отображения анимации загрузки. Рисуемый `loading_img` ресурс включен в стартовый код.

```
import androidx.compose.ui.res.painterResource
import androidx.compose.ui.unit.dp
import androidx.compose.foundation.layout.size
import androidx.compose.foundation.Image

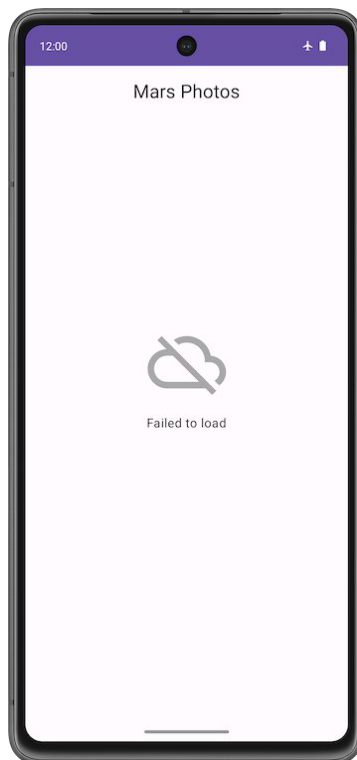
@Composable
fun LoadingScreen(modifier: Modifier = Modifier) {
    Image(
        modifier = modifier.size(200.dp),
        painter = painterResource(R.drawable.loading_img),
        contentDescription = stringResource(R.string.loading)
    )
}
```

10. Под `LoadingScreen` добавьте следующую составную функцию `ErrorScreen`, чтобы приложение могло отображать сообщение об ошибке.

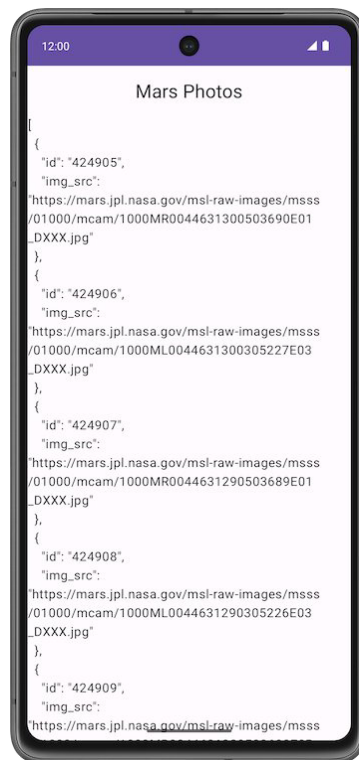
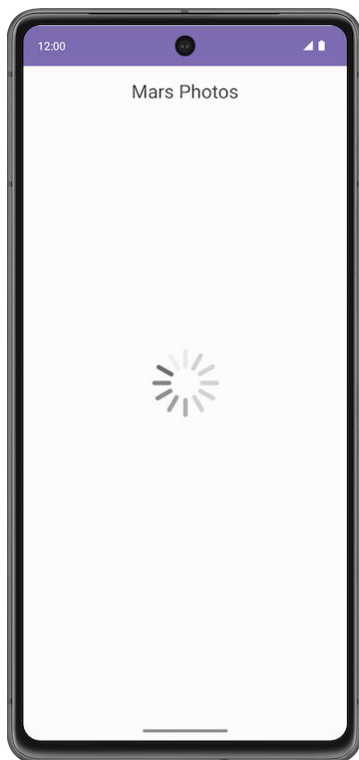
```
import androidx.compose.foundation.layout.Arrangement
import androidx.compose.foundation.layout.Column
import androidx.compose.foundation.layout.padding

@Composable
fun ErrorScreen(modifier: Modifier = Modifier) {
    Column(
        modifier = modifier,
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {
        Image(
            painter = painterResource(id = R.drawable.ic_connection_error), contentDescription = ""
        )
        Text(text = stringResource(R.string.loading_failed), modifier = Modifier.padding(16.dp))
    }
}
```

11. Запустите приложение еще раз, включив режим полета. На этот раз приложение не закрывается внезапно и отображает следующее сообщение об ошибке:



12. Выключите режим полета на телефоне или эмуляторе. Запустите и протестируйте свое приложение, чтобы убедиться, что все работает правильно и вы можете видеть строку JSON.



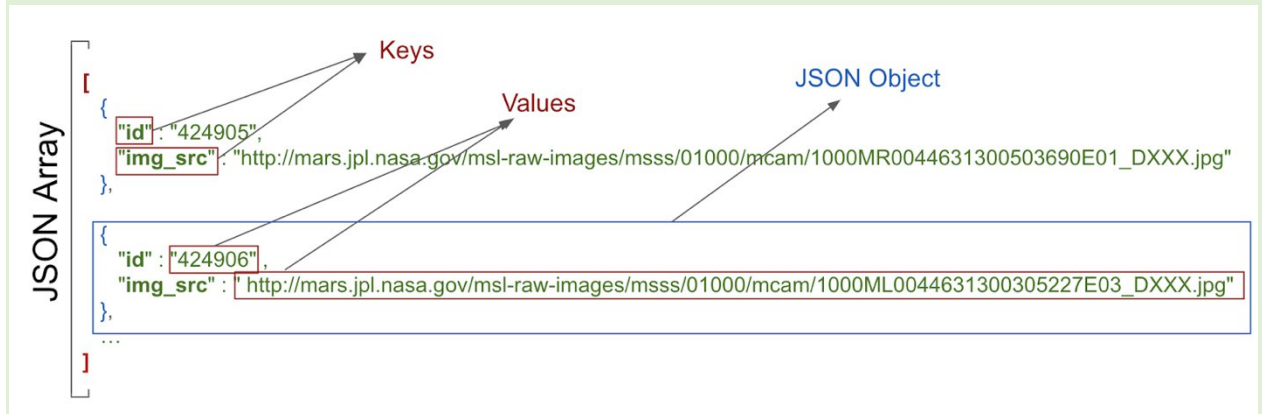
8. Разберите ответ JSON с помощью `kotlinx.serialization`.

JSON

Запрошенные данные обычно форматируются в одном из распространенных форматов данных, например XML или JSON. Каждый вызов возвращает структурированные данные, и вашему приложению необходимо знать, что это за структура, чтобы прочитать данные из ответа.

Например, в этом приложении вы получаете данные с [сервера https://android-kotlin-fun-mars-server.appspot.com/photos](https://android-kotlin-fun-mars-server.appspot.com/photos). Когда вы вводите этот URL-адрес в браузере, вы видите список идентификаторов и URL-адресов изображений поверхности Марса в формате JSON!

Структура примера ответа JSON



Структура ответа JSON имеет следующие особенности:

- Ответ JSON представляет собой массив, обозначенный квадратными скобками. Массив содержит объекты JSON.
- Объекты JSON заключаются в фигурные скобки.
- Каждый объект JSON содержит набор пар ключ-значение, разделенных запятой.
- Двоеточие разделяет ключ и значение в паре.
- Имена заключаются в кавычки.
- Значениями могут быть числа, строки, логические значения, массив, объект (объект JSON) или значение NULL.

Например, `img_src` это URL-адрес, который представляет собой строку. Когда вы вставляете URL-адрес в веб-браузер, вы видите изображение поверхности Марса.

https://mars.nasa.gov/msl-raw-images/msss/01000/mcam/1000MR0



Теперь в вашем приложении вы получаете ответ в формате JSON от веб-сервиса Mars, и это отличное начало. Но для отображения изображений вам действительно нужны объекты Kotlin, а не большая строка JSON. Этот процесс называется *десериализацией*.

Сериализация — это процесс преобразования данных, используемых приложением, в формат, который можно передавать по сети. В отличие от *сериализации*, *десериализация* — это процесс чтения данных из внешнего источника (например, сервера) и преобразования их в объект времени выполнения. Оба они являются важными компонентами большинства приложений, которые обмениваются данными по сети.

Предоставляет `kotlinx.serialization` наборы библиотек, которые преобразуют строку JSON в объекты Kotlin. Существует разработанная сообществом сторонняя библиотека, которая работает с Retrofit, [Kotlin Serialization Converter](#).

В этой задаче вы используете библиотеку `kotlinx.serialization` для анализа ответа JSON от веб-службы на полезные объекты Kotlin, представляющие фотографии Марса. Вы меняете приложение так, чтобы вместо отображения необработанного JSON приложение отображало количество возвращенных фотографий Марса.

Добавить `kotlinx.serialization` зависимости библиотеки

1. Открыть `build.gradle.kts` (Module :app).
2. В блоке `plugins` добавьте плагин. **Внимание:** если система предложит более свежую версию плагина, то укажите свежую (щелкните по строке и если есть новая, то вам высветится окошко с предложением обновить).

```
id("org.jetbrains.kotlin.plugin.serialization") version "1.9.10"
```

3. В этом разделе `dependencies` добавьте следующий код, чтобы включить зависимость `kotlinx.serialization`. Эта зависимость обеспечивает сериализацию JSON для проектов Kotlin.

```
// Kotlin serialization
implementation("org.jetbrains.kotlin:kotlinx-serialization-json:1.6.0")
```

4. Найдите строки для скалярного преобразователя Retrofit в блоке `dependencies` и измените его на использование `kotlinx-serialization-converter`:

Замените следующий код

```
// Retrofit with scalar Converter
implementation("com.squareup.retrofit2:converter-scalars:2.9.0")
```

со следующим кодом

```
// Retrofit with Kotlin serialization Converter

implementation("com.jakewharton.retrofit:retrofit2-kotlinx-serialization-converter:1.0.0")
implementation("com.squareup.okhttp3:okhttp:4.11.0")
```

5. Нажмите **«Sync Now»**, чтобы перестроить проект с новыми зависимостями.

Примечание. В проекте могут отображаться ошибки компилятора, связанные с удаленной скалярной зависимостью Retrofit. Вы исправите их на следующих шагах.

Реализуйте класс данных Mars Photo.

Пример записи ответа JSON, который вы получаете от веб-службы, выглядит примерно следующим образом, аналогично тому, что вы видели ранее:

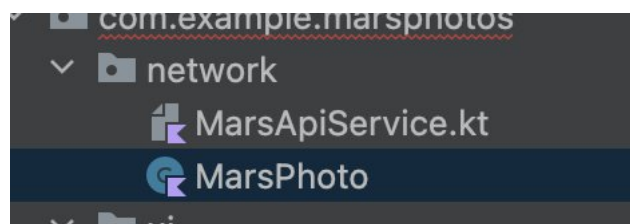
```
[
  {
    "id": "424906",
    "img_src": "http://mars.jpl.nasa.gov/msl-raw-images/msss/01000/mcam/1000ML0044631300305227E03_DXXX.jpg"
  },
  ...]
```

Обратите внимание, что в приведенном выше примере каждая запись фотографии Марса имеет следующие пары ключей и значений JSON:

- `id`: идентификатор свойства в виде строки. Поскольку он заключен в кавычки (" "), он имеет тип `String`, а не `Integer`.
- `img_src`: URL-адрес изображения в виде строки.

`kotlinx.serialization` анализирует эти данные JSON и преобразует их в объекты Kotlin. Для этого у `kotlinx.serialization` должен быть класс данных Kotlin для хранения результатов анализа. На этом этапе вы создаете класс данных `MarsPhoto`.

1. Щелкните правой кнопкой мыши пакет **network** и выберите **«New» > «Class/file Kotlin»**.
2. В диалоговом окне выберите **Class** и введите имя класса `MarsPhoto`. Это действие создает новый файл с именем `MarsPhoto.kt` в пакете `network`.



3. Добавив ключевое слово `data` перед определением класса, превратим `MarsPhoto` в класс данных. Замените фигурные скобки `{}` на круглые `()`. Это изменение приводит к ошибке, поскольку в классах данных должно быть определено хотя бы одно свойство.

```
data class MarsPhoto()
```

5. Добавьте следующие свойства в определение класса `MarsPhoto`.

```
data class MarsPhoto(  
    val id: String,  
    val img_src: String  
)
```

6. Чтобы сделать класс `MarsPhoto` сериализуемым, аннотируйте его с помощью `@Serializable`.

```
import kotlinx.serialization.Serializable
```

```
@Serializable
```

```
data class MarsPhoto(  
    val id: String,  
    val img_src: String  
)
```

Обратите внимание, что каждая переменная в классе `MarsPhoto` соответствует имени ключа в объекте JSON. Чтобы сопоставить типы в нашем конкретном ответе JSON, вы используете `String` объекты для всех значений.

При анализе JSON `kotlinx.serialization` сопоставляет ключи по имени и заполняет объекты данных соответствующими значениями.

Аннотация `@SerialName`

Иногда имена ключей в ответе JSON могут запутать свойства Kotlin или не соответствовать рекомендуемому стилю кодирования. Например, в файле JSON ключ `img_src` использует подчеркивание, тогда как в соглашении Kotlin для свойств используются буквы верхнего и нижнего регистра (верблюжий регистр).

Чтобы использовать имена переменных в вашем классе данных, которые отличаются от имен ключей в ответе JSON, используйте аннотацию `@SerialName`. В следующем примере имя переменной в классе данных — `imgSrc`. Переменную можно сопоставить с атрибутом JSON `img_src`, используя `@SerialName(value = "img_src")`.

1. Замените строку ключа `img_src` на строку, показанную ниже.

```
import kotlinx.serialization.SerialName
```

```
@SerialName(value = "img_src")  
val imgSrc: String
```

Обновите `MarsApiService` и `MarsViewModel`.

В этой задаче вы будете использовать конвертер `kotlinx.serialization` для преобразования объекта JSON в объекты Kotlin.

1. Открыть `network/MarsApiService.kt`.
2. Обратите внимание на неразрешенные ошибки ссылок для `ScalarsConverterFactory`. Эти ошибки являются результатом изменения зависимости Retrofit, описанного в предыдущем разделе.
3. Удалите импорт для `ScalarConverterFactory`. Вторую ошибку вы исправите позже.

Удалять:

```
import retrofit2.converter.scalars.ScalarsConverterFactory
```

4. В объявлении объекта `retrofit` измените построитель Retrofit, чтобы использовать `kotlinx.serialization` вместо `ScalarConverterFactory`.

```
import com.jakewharton.retrofit2.converter.kotlinx.serialization.asConverterFactory
import kotlinx.serialization.json.Json
import okhttp3.MediaType.Companion.toMediaType
```

```
private val retrofit = Retrofit.Builder()
    .addConverterFactory(Json.asConverterFactory("application/json".toMediaType()))
    .baseUrl(BASE_URL)
    .build()
```

Примечание. Если Android Studio показывает какое-либо предупреждение `kotlinx.serialization.ExperimentalSerializationApi`, игнорируйте его. Вам не нужно беспокоиться об этом предупреждении. Эта проблема будет решена в более поздних версиях Kotlin.

```
private val retrofit = Retrofit.Builder() Retrofit.Builder
    .addConverterFactory(Json.asConverterFactory(MediaType.get("application/json"))) Retrofit.Builder
    .baseUrl(BASE_URL)
    .build()
```

This declaration needs opt-in. Its usage should be marked with '@kotlinx.serialization.ExperimentalSerializationApi' or '@OptIn(kotlinx.serialization.ExperimentalSerializationApi::class)'

Opt in for 'ExperimentalSerializationApi' on 'retrofit'   More actions...  

Теперь, когда все готово, вы можете попросить Retrofit вернуть список объектов `MarsPhoto` из массива JSON вместо возврата строки JSON.

5. Обновите интерфейс Retrofit `MarsApiService`, чтобы он возвращал список объектов `MarsPhoto` вместо возврата файла `String`.

```
interface MarsApiService {
    @GET("photos")
    suspend fun getPhotos(): List<MarsPhoto>
}
```

6. Внесите аналогичные изменения в файл `viewModel`. Откройте `MarsViewModel.kt` и прокрутите вниз до метода `getMarsPhotos()`.

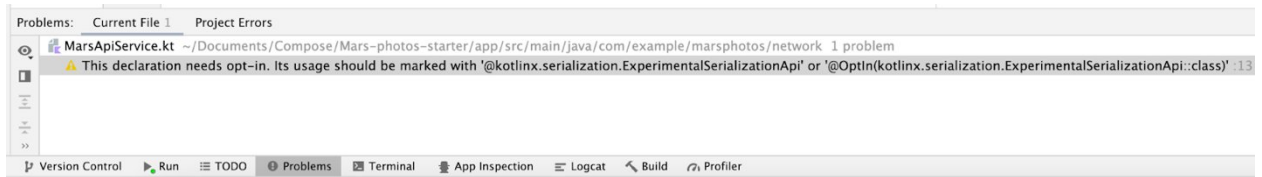
В методе `getMarsPhotos()` результатом будет `List<MarsPhoto>`, а не `String`. Размер этого списка равен количеству фотографий, которые были получены и проанализированы.

7. Чтобы распечатать количество полученных фотографий, обновите их `marsUiState` следующим образом:

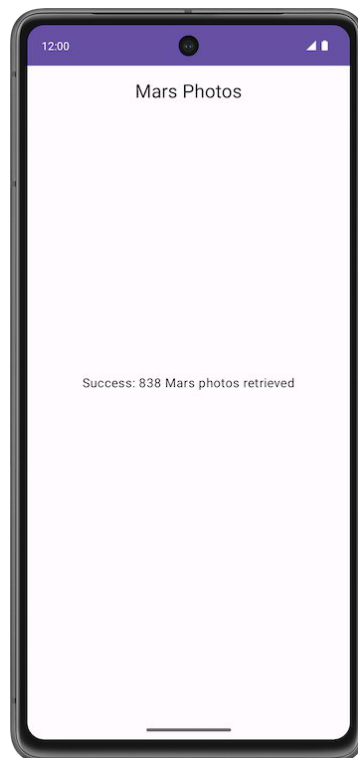
```
val listResult = MarsApi.retrofitService.getPhotos()
MarsUiState.Success(
    "Success: ${listResult.size} Mars photos retrieved"
)
```

8. Убедитесь, что режим полета отключен на вашем устройстве или эмуляторе. Скомпилируйте и запустите приложение.

Примечание . В Android Studio вы получите предупреждение (на вкладке «Проблемы») о том, что декларация `ExperimentalSerializationApi` требует согласия. Поскольку этот API будет стабильным в будущих выпусках Kotlin, это предупреждение можно смело игнорировать.



На этот раз сообщение должно показывать количество свойств, возвращаемых веб-сервисом, а не большую строку JSON:



Примечание. Если ваше подключение к Интернету не работает, обязательно отключите режим полета на своем устройстве или в эмуляторе.

10. Резюме

Веб-сервисы REST

- Веб-сервис — это программная функциональность, предлагаемая через Интернет, которая позволяет вашему приложению отправлять запросы и получать данные обратно.
- Общие веб-сервисы используют архитектуру [REST](#). Веб-сервисы, предлагающие архитектуру REST, известны как сервисы *RESTful*. Веб-сервисы RESTful создаются с использованием стандартных веб-компонентов и протоколов.
- Вы делаете запрос к веб-службе REST стандартизированным способом через URI.
- Чтобы использовать веб-службу, приложение должно установить сетевое соединение и связаться со службой. Затем приложение должно получить и проанализировать данные ответа в формате, который может использовать приложение.
- Библиотека [Retrofit](#) — это клиентская библиотека, которая позволяет вашему приложению отправлять запросы к веб-службе REST.
- Используйте преобразователи, чтобы сообщить Retrofit, что делать с данными, которые он отправляет веб-службе и получает обратно от веб-службы. Например, он [ScalarsConverter](#) рассматривает данные веб-службы как тот [String](#) или иной примитив.
- Чтобы ваше приложение могло подключаться к Интернету, добавьте `"android.permission.INTERNET"` разрешение в манифест Android.
- Ленивая инициализация делегирует создание объекта при первом его использовании. Он создает ссылку, но не объект. Когда к объекту обращаются впервые, ссылка создается и используется каждый раз в дальнейшем.

Анализ JSON

- Ответ веб-службы часто форматируется в [JSON](#) — распространенном формате для представления структурированных данных.
- Объект JSON представляет собой набор пар ключ-значение.
- Коллекция объектов JSON представляет собой массив JSON. Вы получаете массив JSON в качестве ответа от веб-службы.
- Ключи в паре ключ-значение заключены в кавычки. Значения могут быть числами или строками.
- В Kotlin инструменты сериализации данных доступны в отдельном компоненте [kotlinx.serialization](#). [kotlinx.serialization](#) предоставляет наборы библиотек, которые преобразуют строку JSON в объекты Kotlin.
- Существует разработанная сообществом библиотека преобразователя сериализации Kotlin для Retrofit: [Retrofit2-kotlinx-serialization-converter](#).
- [kotlinx.serialization](#) сопоставляет ключи в ответе JSON со свойствами объекта данных с тем же именем.
- Чтобы использовать другое имя свойства для ключа, добавьте к этому свойству аннотацию `@SerializedName` и ключ JSON `value`.

11. Узнать больше

Документация для разработчиков Android:

- [Руководство по архитектуре приложения | Android-разработчики](#)
- [Обзор модели представления](#)
- [ViewModelScope](#)

Документация Котлина:

- [Исключения: попробовать, поймать, наконец, бросить, Ничего.](#)
- [Сопрограммы, официальная документация](#)
- [Контекст сопрограммы и диспетчеры](#)
- [Сериализация | Котлин](#)

Другой:

- [Модернизация](#)
- [Retrofit 2 Converter.Factory для сериализации Kotlin](#)